

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

CSE 3221 Parallel Computer Architecture and Programming

Project Synopsis on

“CUDA Image Processing : Grayscale, Edge Detection & Blurring “

SUBMITTED BY

Pranamya G Kulal (8) 220905018
Ayush Prabhu (6) 220905010
Tanish Sunilkumar (56) 220905532
Ranjith G Rao (9) 220905020
Bhavya Sri Burri (43) 220905386

Section A

Under the Guidance of:
Dr. Radhakrishna Bhat
Department of Computer Science and Engineering
Manipal Institute of Technology, Manipal, Karnataka – 576104

2024-25

Abstract

Image processing is a vital field in computer science, enabling the extraction and manipulation of meaningful data from images for various applications, including computer vision and multimedia analysis. With the advent of CUDA (Compute Unified Device Architecture), GPUs have become powerful tools for parallel computation, significantly accelerating image processing tasks. This project focuses on implementing three fundamental image processing techniques—grayscale conversion, edge detection, and blurring—using CUDA to leverage GPU parallelism for enhanced performance.

The primary objective is to demonstrate the efficiency of GPU-based parallel computing in image processing compared to traditional CPU-based sequential methods. The traditional CPU-based implementations of these algorithms, though straightforward, suffer from performance bottlenecks when dealing with high-resolution images or real-time data streams. CUDA allows us to overcome these limitations by enabling the execution of thousands of threads in parallel, distributing pixel-level operations across GPU cores for faster computation.

For grayscale conversion, each pixel is transformed using a weighted sum of RGB components, handled independently by individual threads. In edge detection, we implemented custom CUDA kernels mimicking Sobel operators to highlight regions of high spatial frequency, which typically correspond to edges. Additionally, a blurring operation using a simple box filter demonstrates spatial convolution, where each output pixel is calculated based on its neighborhood

In conclusion, this project demonstrates how CUDA-based parallel computing can optimize image processing workflows, paving the way for scalable solutions in real-world applications such as medical imaging, satellite imagery analysis, and autonomous systems.The experience gained here lays a strong foundation for more advanced work in computer vision and high-performance computing.

Introduction

Image Processing is a method to perform operations on an image to enhance it or extract useful information. It's a type of signal processing where the input is an image, and the output can be:

- Image which is enhanced like making it clearer or brighter, or
- Extracted information from the image like detecting faces or objects.

The fundamental operations in image processing can be grayscale conversion, blurring, and edge detection — each serving a unique role in preparing images for further analysis or improving their clarity.

- Grayscale Conversion simplifies by removing color information from an image and retaining only brightness (intensity) values. This reduces computational complexity and is often the first step in many vision algorithms.
- Blurring, such as Gaussian blur, softens an image by averaging nearby pixels which is commonly used to reduce noise and minor details, helping highlight more significant structures in that image.
- Edge Detection finds boundaries between different regions in an image. By detecting sharp intensity changes, edge detection algorithms help in identifying shapes, objects, and textures.

These operations are widely used in applications ranging from medical imaging and surveillance to self-driving cars and augmented reality.

Proposed Methodology

This section contains an elaborate approach to implement grayscale conversion, edge detection and image blurring using CUDA in a step by step manner. Google Colab has been used to execute the required program as it provides free access to GPU's.

1) Setting Up the Environment in Google Colab
- Navigate to Runtime > Change runtime type > GPU to enable CUDA support.
- Verify GPU availability using !nvidia-smi
- Check if CUDA is installed !nvcc - - version

2) Installing and Setting Up Required Libraries
- Install OpenCV and NumPy !pip install opencv-python numpy
- Import image files to Colab's workspace

3) Loading and Preprocessing the Image
- Use OpenCV to read and convert images into arrays suitable for CUDA processing
- Convert images into flattened 1D array for GPU processing

4) Implement CUDA Kernels for Image Processing
CUDA Kernels are written in a separate .cu file
- Grayscale conversion: Compute weighted sum of RGB values.
- Edge detection: Apply Sobel filter for detecting intensity changes.
- Image blurring: Perform convolution.

Compile with nvcc and run the compiled executable

5) Transferring Data Between CPU and GPU
- Allocate memory on the GPU (cudaMalloc) and copy image data (cudaMemcpy).
- Launch CUDA kernels with optimized block and grid sizes.
- Retrieve the processed image back from the GPU to CPU memory.

6) Comparing CPU and GPU Performance
Measure execution time using cudaEvent_t

7) Visualizing Results
Convert processed image back to OpenCV format and display it in C